Buffer Overflow and Format String Attacks

Dr. Vivek Balachandran

Vivek.b@singaporetech.edu.sg

Overview

- Memory management
- Buffer Overflow
- Format String Vulnerability



Memory management in C/C++

SINGAPORE INSTITUTE OF TECHNOLOGY

- Program responsible for its memory management
  - Very error-prone
  - Who here has had a C/C++ program crash with a segmentation fault?
- Typical bugs
  - Writing past the bound of an array
  - Pointer trouble
    - Missing initialization, bad pointer arithmetic, use after free, double free, memory leaks etc.

What is a buffer overflow?

Suppose in a C program we have an array of length 4

```
char buffer[4];
```

What happens if we execute the statement below?

```
buffer[4] = ‘a’;
```

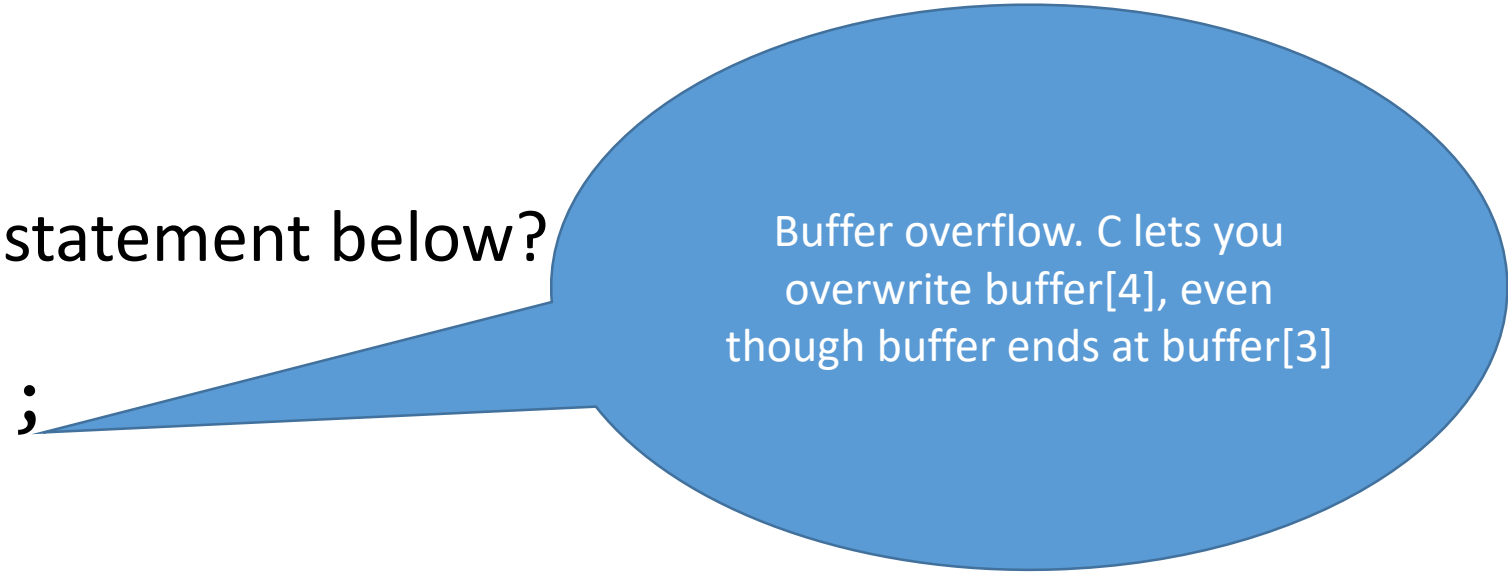
What is a buffer overflow?

Suppose in a C program we have an array of length 4

```
char buffer[4];
```

What happens if we execute the statement below?

```
buffer[4] = 'a';
```



If the data written is user input that can be controlled by an attacker, this vulnerability can be exploited.

Buffer overflow

- A buffer overflow is a bug that affects low-level code, typically in C and C++, with significant security implications
- Normally, a program with this bug will simply crash
- But attacker can alter the situation that causes the program to do much worse
  - Steal private information (e.g Heartbleed)
  - Corrupt valuable information
  - Run code of the attacker's choice

Why are they important?

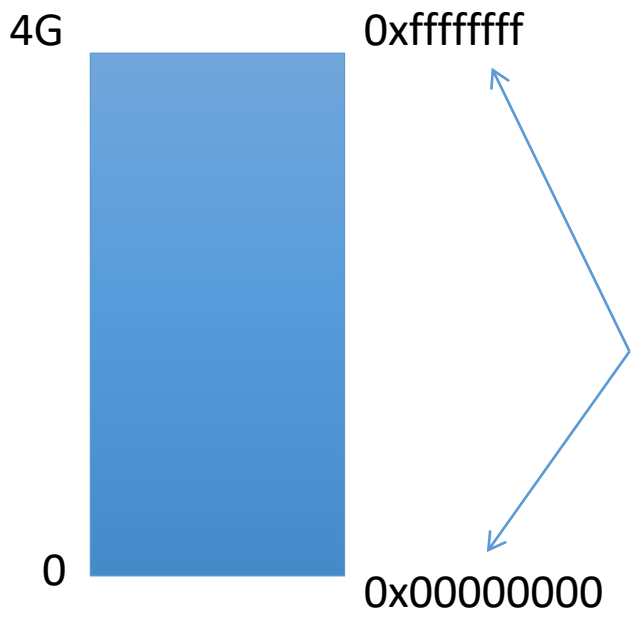
- C and C++ are still popular
  - Microsoft Windows
  - Microsoft Excel
  - Linux
  - Mozilla
  - MySQL
- Critical systems are written in C/C++
  - OS kernels and utilities
  - High-performance servers like Apache, ngnix etc.
  - Embedded systems

Memory Layout

- How is program data laid out in memory?
- What does the stack look like?
- What effect does calling and returning from a function have on memory?
- Assume a Linux process model
  - Similar to other operating systems

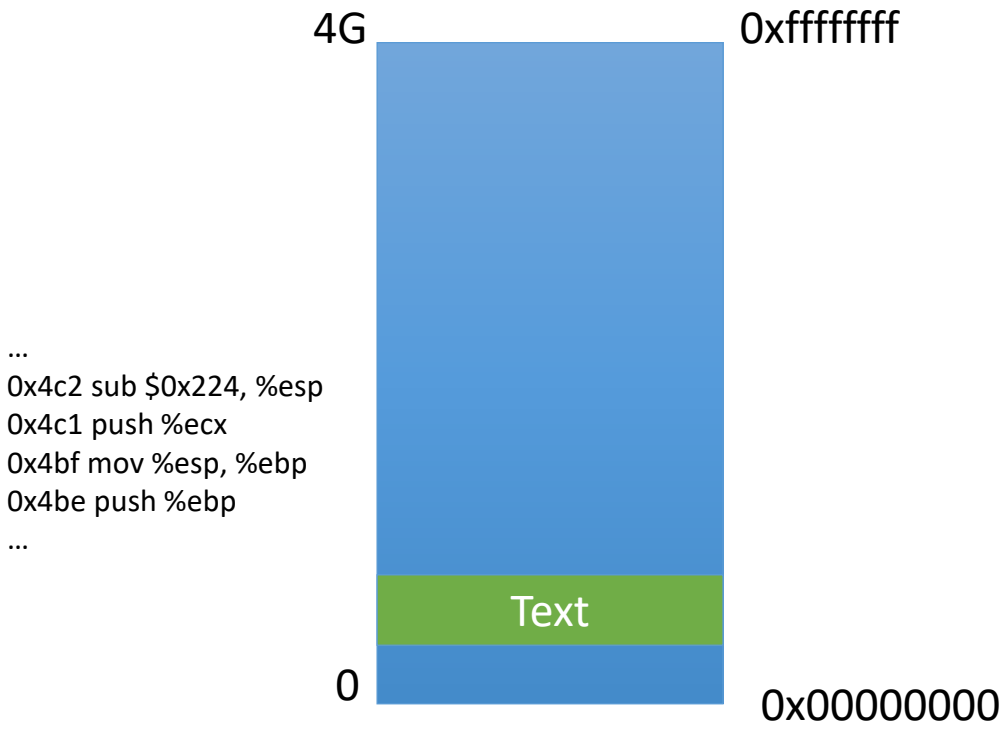
Memory

The process's view of memory is that it owns all of it

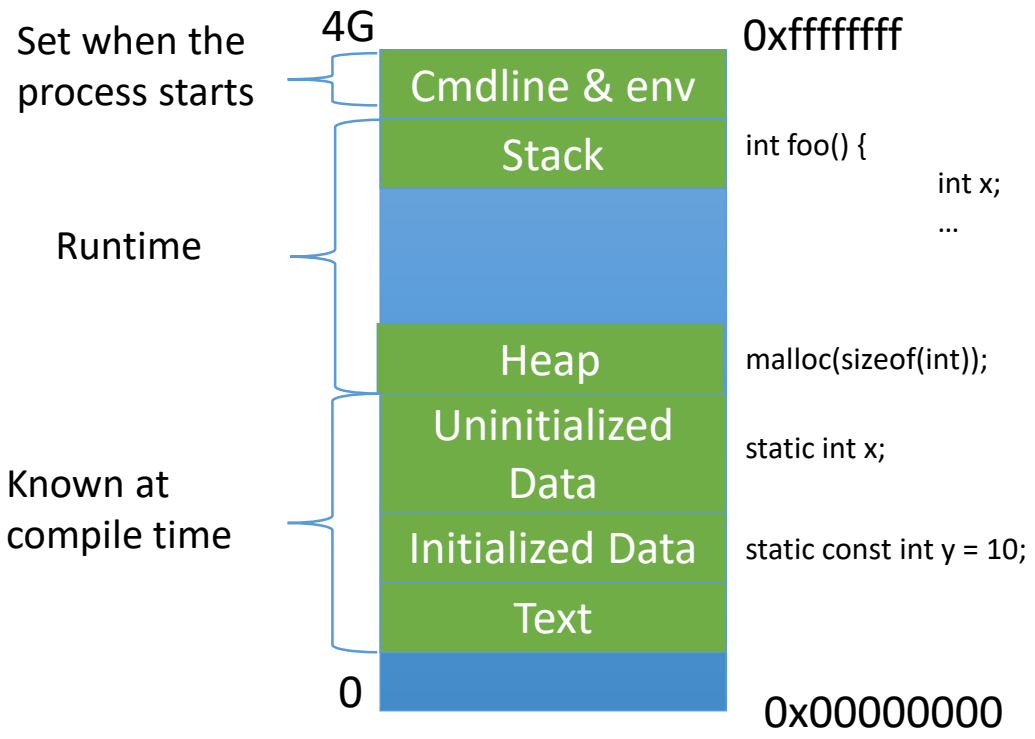


In reality, these are virtual addresses; the OS/CPU maps them to physical addresses

Instructions in Memory

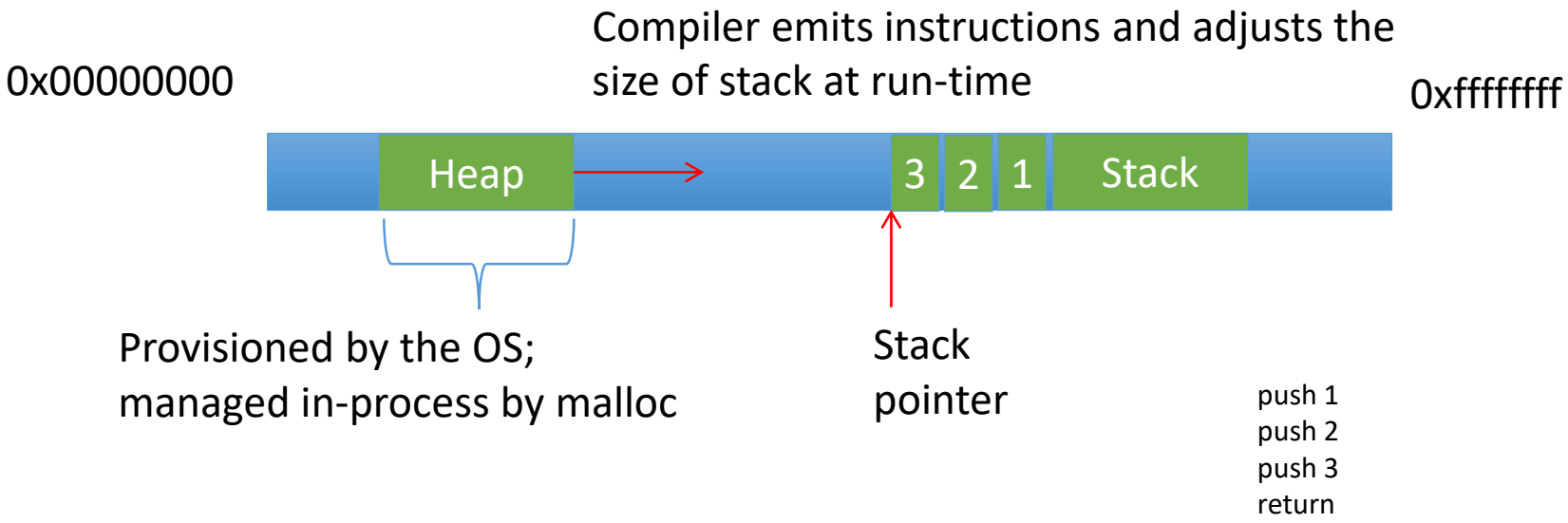


Data areas


Set when the process starts
Runtime
Known at compile time
4G
0
0xffffffff
0x00000000
Cmdline & env
Stack
Heap
Uninitialized Data
Initialized Data
Text
int foo() {
int x;
…
malloc(sizeof(int));
static int x;
static const int y = 10;

Memory allocation

Stack and heap grow in opposite directions



Function Calls

SINGAPORE INSTITUTE OF TECHNOLOGY

- What happens when we call a function?
  - What data needs to be stored?
  - Where does it go?
- What happens when we return from a function?
  - What data needs to be restored?
  - Where does it come from?

Function Calls

SINGAPORE INSTITUTE OF TECHNOLOGY

- What happens when we call a function?
  - What data needs to be stored?
    - Function arguments
    - Return address
    - Base pointer
  - Where does it go?
    - Stack
- What happens when we return from a function?
  - What data needs to be restored?
    - Restore stack state
  - Where does it come from?
    - Stack

Basic stack layout

```
void foo(char *arg1, int arg2, int arg3)
{
        char loc1[4];
        int loc2;
        …
}
```

0xffffffff

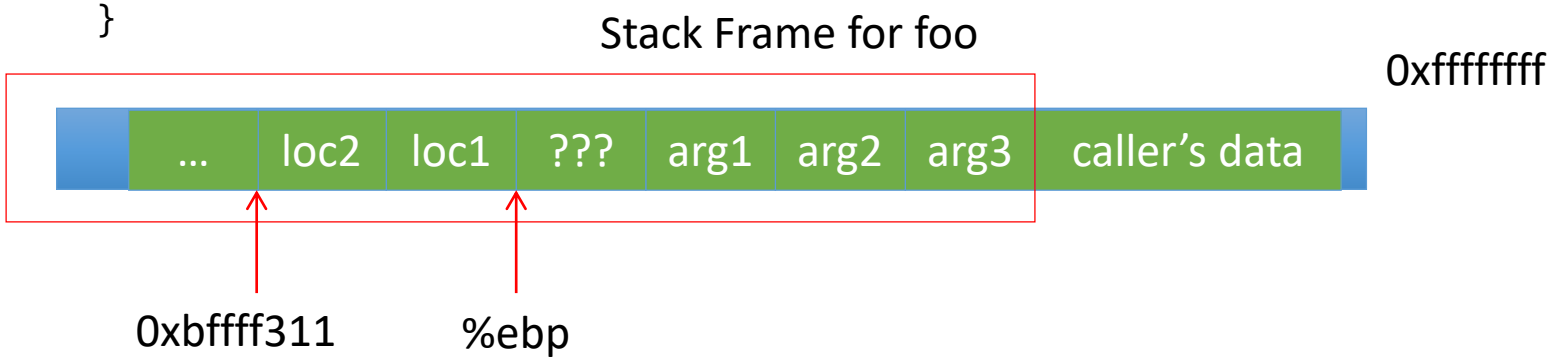


Local variables pushed in same order as they appear (ultimately up to the compiler may not be allocated or stored only in registers based on optimization level used)

Arguments pushed in reverse order

Accessing a variable

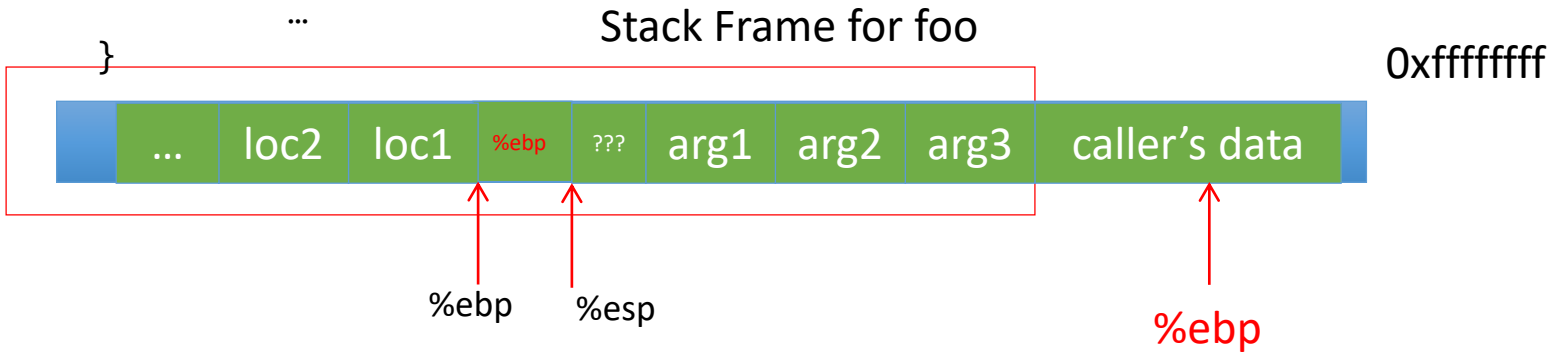
```
void foo(char *arg1, int arg2, int arg3)
{
        …
        loc2++;   // Where is loc2?
        …
}
```



Can't know the absolute address at compile time, but the relative address of loc2 is always 8B before frame pointer (or base pointer ebp)

Setting the stack for the callee fucntion

```
void main(char *arg1, int arg2, int arg3)
{
        …
        foo("hello world!", 1, 2)   //How do we restore
%ebp?
        …
}
```

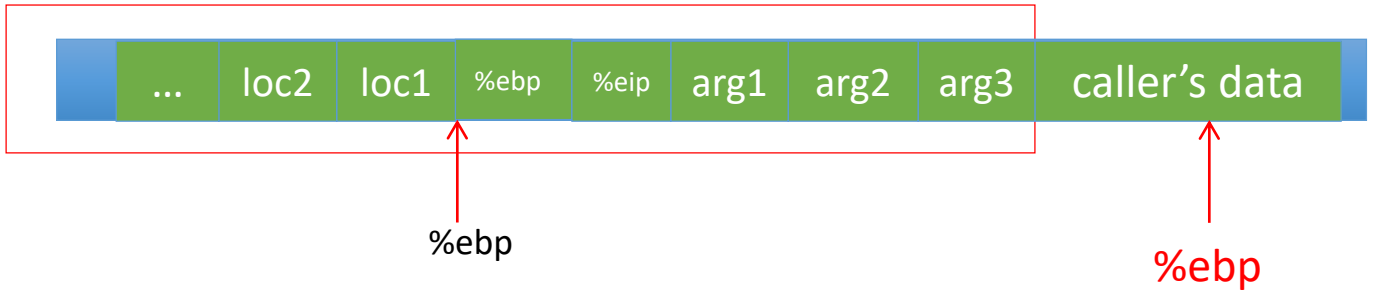


1. Push %ebp
   - Backing up caller function's base pointer in stack
2. Set %ebp to current %esp
   - %esp is changed due to step 1
   - Set the top of stack as the current base pointer for the callee function
3. Set %ebp to (%ebp) at return
   - Restoring the backed up %ebp from step 1

Return from Functions

```
void main(char *arg1, int arg2, int arg3)
{
        …
        foo("hello world!", 1, 2)
                //How do we resume here?
        …
}
```

Stack Frame for foo



Set %eip to 4 (%ebp) at return

Push next %eip before call

x86 Calling Conventions

- Calling function
  - Push arguments onto the stack in reverse
  - Push the return address
  - Jump to the function's address
- Callee function
  - Push old frame pointer onto the stack (%ebp)
  - Set frame pointer (%ebp) to where the end of stack is right now (%esp)
  - Push local variables onto the stack
- Returning function
  - Reset the previous stack frame: %esp = %ebp, %ebp = (%ebp)
  - Jump back to return address: %eip = 4(%esp)

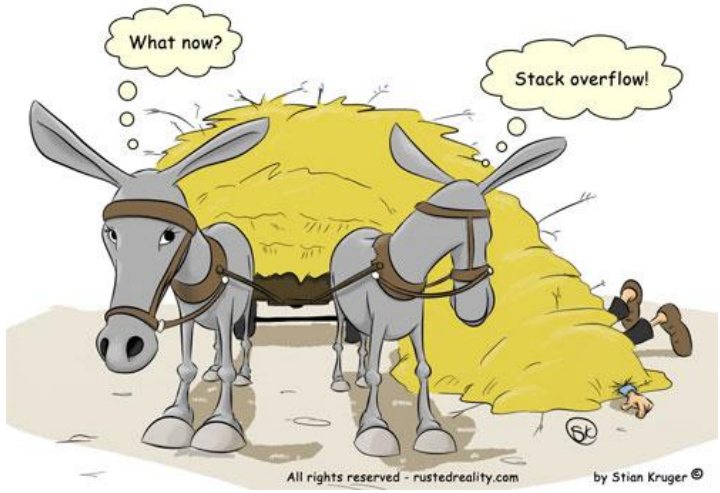


"Can I call you back? I've got a dinner to go to."

Stack overflow

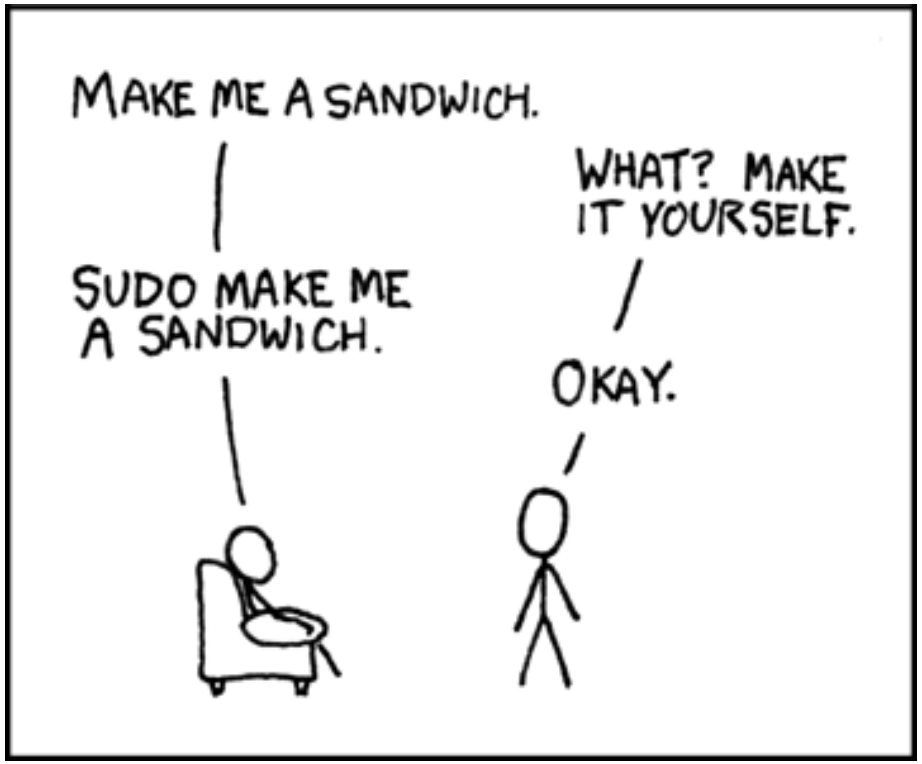
```
void f(int x) {
        char[8] buf;
        gets(buf);
}
void main() {
        f(…); …
}
void format_hard_disk(){…}
```

What if gets() reads more than 8 bytes ?





Demo – Buffer Overflow



Format string attacks

- Another way of corrupting the stack
  - Strings can contain special characters in C, e.g. %s
    - printf("Cannot find file %s", filename);
    - Such strings are called format strings
  - What happens if we execute the code below?
    - printf("Cannot find file %s");
  - What may happen if we execute
    - printf(string) where string is user-supplied?

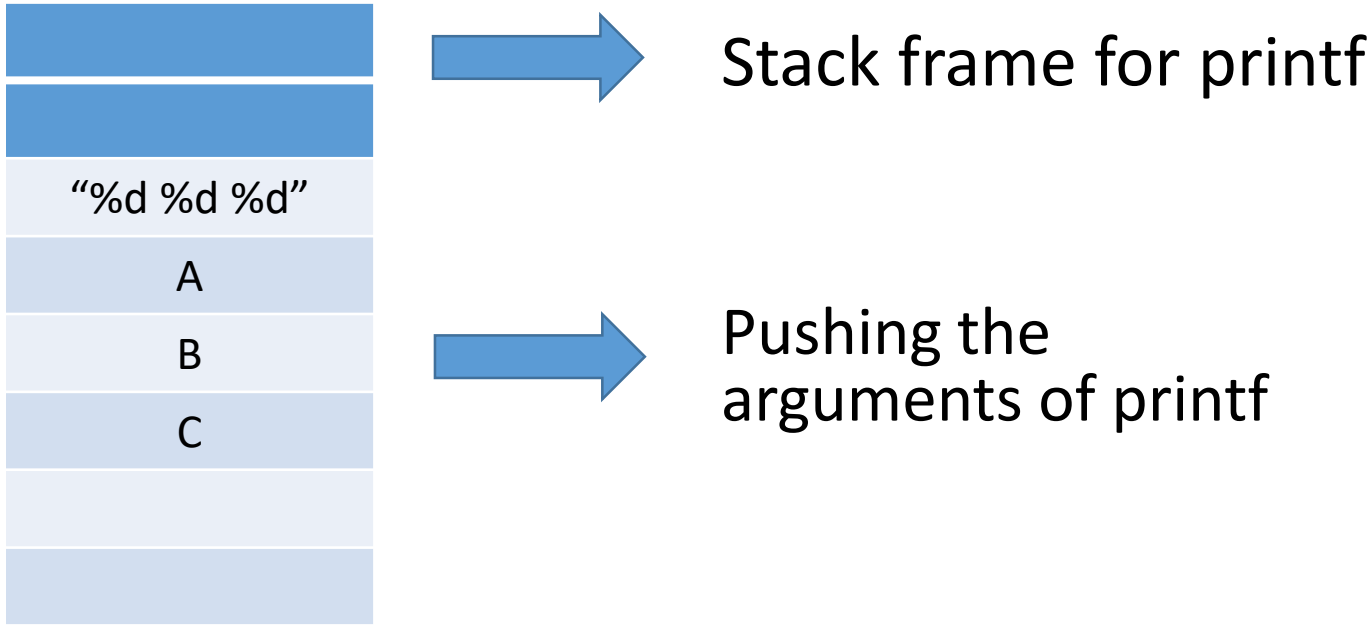
Format string attacks

- %x reads and prints some bytes from stack
  - This can leak sensitive data
  - E.g. printf(s); will dump the stack if attacker supplies as input the string

%x %x %x %

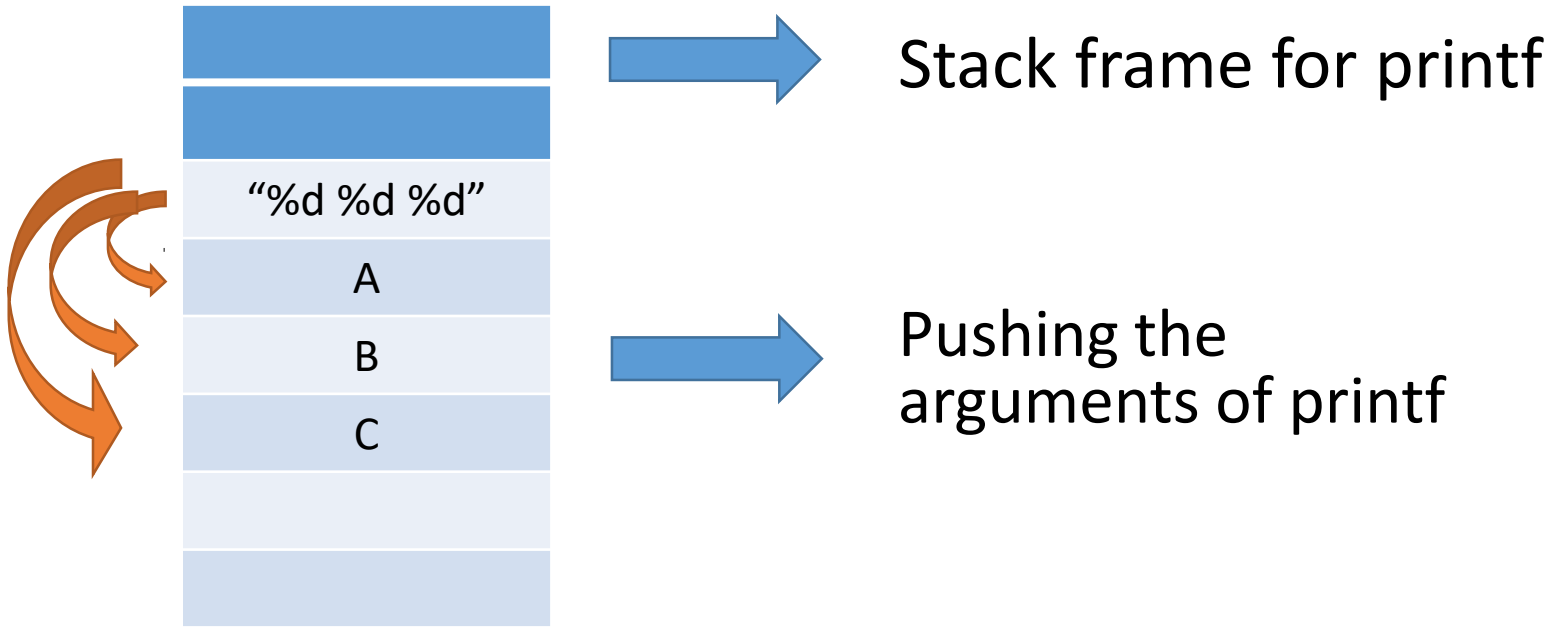
Format String Attack – How it looks in the stack

```
foo(){
    printf("%d %d %d", a, b, c);
}
```



Format String Attack – How it looks in the stack

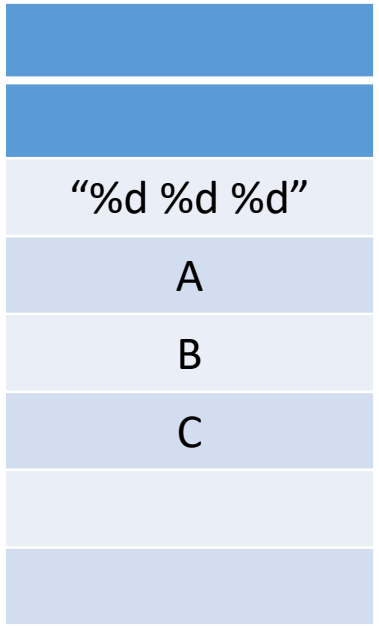
```
foo(){
    printf("%d %d %d", a, b, c);
}
```



Format String Attack – How it looks in the stack

```
foo(){
    printf("%d %d %d", a, b, c);
}
```

What happens when there is no a, b and c as arguments in printf()



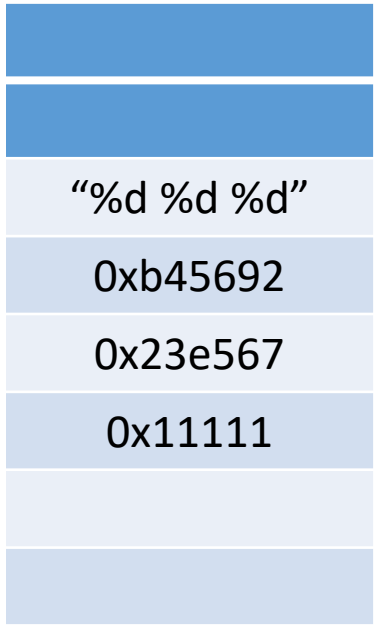
Stack frame for printf

Pushing the arguments of printf

Format String Attack – How it looks in the stack

```
foo(){
    printf("%d %d %d");
}
```

Printf will take whatever values that are available on the stack



Stack frame for printf

Print these as a,b,c

Format string attacks

- What makes it more potent
  - Ability of printf format strings to write to the stack
- %n writes the number of characters printed so far onto the stack
  - This allows the attacker to corrupt the stack

- You can combine them to create advanced format string attacks

```c
#include <stdio.h>

main()
{
        int a,b;
        printf("1234567%n",&a);
        printf("The value in a = %d\n",a);

}
```



```
1234567The value in a = 7
```

Advanced format string attacks

- An attacker can try to control which address is used to reading from memory using %s or for writing to memory using %n with specially crafted format strings
  - \xEF\xCD\xCD\xAB %x %x ... %x %s with the right number of %x characters, this will print the string located at address ABCDCDEF
  - \xEF\xCD\xCD\xAB %x %x ... %x %n with the right number of %x characters, this will write the number of characters printed so far to location ABCDCDEF

An easy fix

- Format string attacks are easy to stop and fix:
  - Replace printf(str)
  - With printf(“%s”, str)

Demo – Format String Attack

